# Install the AI Demo

Three ways to run the Ping AI demo stack. Pick the first unless you need your own isolated instance or must work offline.

## Choose your tier

| TIER | BEST FOR | YOU INSTALL | ISOLATION |
|---|---|---|---|
| **1 · Shared demo** ai-demo.ping-devops.com | Walkthroughs, trying the demo, showing a customer | A browser | Shared with everyone — demo state is common |
| **2 · Your own namespace** <slug>.ai-demo.ping-devops.com | Developing the demo, running it for a specific audience, parallel sessions | Repo clone + kubectl + SE cluster access | Fully yours; the canonical URL stays untouched |
| **3 · Your laptop** localhost via port-forwards | Offline work, no cluster access, deep debugging | OrbStack + Homebrew (~23 GB images pulled once) | Private; runs offline after first install |

## Tier 1 — use the shared demo

Open https://ai-demo.ping-devops.com and sign in with your Ping SSO (@pingidentity.com or @pingone.com email). First visit: you'll be redirected to PingOne to authenticate, then back to the app.

> **If sign-in is refused:** your email's domain is in the allowlist (pingidentity.com, pingone.com), but access also requires membership of the demo access group in PingOne. Contact the demo owner (cmuir) to be added, then sign in again.

## Tier 2 — deploy your own namespace on the SE cluster

Same cluster as the shared demo, but your own namespace, your own URL, your own state. Nothing you do touches the shared instance.

### Prerequisites

- macOS with **Docker** (OrbStack or Docker Desktop) and **Homebrew**
- **kubectl** + access to the SE cluster (ping-dev-aws-us-east-2): run `./run-pingaws.sh kubeconfig` to install it, or ask the SE DevOps team for the kubeconfig

- **GitHub CLI** authenticated ( `gh auth login` ) — with access to the AI-DEMO2 repo
- Your **Ping email** (it names your namespace: jsmith@pingidentity.com → ping-devops-jsmith)
- A **PingOne worker app** in your own demo environment — the bootstrap creates one if you have worker credentials for an environment you own

#### Install

```
git clone https://github.com/cmuir_pingcorp/AI-DEMO2 && cd AI-DEMO2
./run-pingaws.sh kubeconfig          # one-time: SE kubeconfig into ~/.kube
./run-pingaws.sh start               # build + push + deploy (~30 min first time)
```

When it finishes, the banner shows your URL: **https://<slug>.ai-demo.ping-devops.com** (derived from your namespace — e.g. jsmith → jsmith.ai-demo.ping-devops.com). No DNS or certificate work is needed; wildcard DNS routes your host to the cluster's shared ingress and cert-manager issues your cert automatically.

#### What runs where

Everything runs in the cluster — your laptop only builds and pushes images. The only host-side state is the repo clone, demo_api_server/.env (PingOne credentials), and mkcert certificates under certs/. Your namespace runs the full stack: app + BFF, agents, MCP servers + gateway, LLM tier (models download in-cluster from HuggingFace), and observability.

#### Day-2 commands

```
./run-pingaws.sh status              # pods in your namespace
./run-pingaws.sh update code bff     # rebuild + redeploy one service
./run-pingaws.sh update pingone      # re-register OAuth redirects (after URL change)
./run-pingaws.sh basic               # scale to dashboard/admin + login only
./run-pingaws.sh undeploy            # delete YOUR namespace's workloads (asks first)
```

> **Your namespace, your URL:** deploys derive `<slug>.ai-demo.ping-devops.com` from your namespace, so you never collide with the shared demo. Set `APP_HOST=` to override the derived hostname if you need a specific one.

## Tier 3 — run on your laptop

Everything the SE cluster hosts, but on your machine via OrbStack's Kubernetes. Use when you can't reach the SE cluster or need to work offline.

#### Install

```
# 1. Install OrbStack and enable Kubernetes in its settings
# 2. Then:
git clone https://github.com/cmuir_pingcorp/AI-DEMO2 && cd AI-DEMO2
./run-k8.sh build                    # ~23 GB of images, one-time
./run-k8.sh                          # deploy + port-forward
```

The demo is then at https://local.ping-devops.com:4000. Models download in-cluster from HuggingFace on first deploy. For 16 GB Macs, add the small overlay: `kubectl apply -f k8s/overlays/small/`.

> **Laptop vs cluster differences:** the LLM tier runs llama.cpp in-cluster on both; on a laptop the MLX (Apple) agent mode can use host-side `mlx_lm.server` instead (optional, see k8s/README.md). Cluster deploys use GHCR images; laptop deploys build locally.

## Troubleshooting

| SYMPTOM | FIX |
|---|---|
| Tier 2 deploy: `HOSTED_ACCESS_REQUIRED_GROUP_ID must be set` | The deploy reads it from demo_api_server/.env. Run the PingOne bootstrap first, or export it (value from the demo owner). |
| Tier 2: your URL doesn't resolve | Wildcard DNS covers `*.ai-demo.ping-devops.com` and `*.ping-devops.com`. If `<slug>.ai-demo.ping-devops.com` fails, check `kubectl get ingress -n ping-devops-<slug>` — the host must match the derived one exactly. |
| Tier 2: cert not issued for the a2a host | The langchain-a2a cert is two labels deep and uses cert-manager (DNS-01). Check `kubectl get certificate -n ping-devops-<slug>`; issuance takes a few minutes on first deploy. |
| Sign-in loops back to login | Your `PUBLIC_APP_URL` doesn't match the ingress host, so OAuth redirects mismatch. Re-run `./run-pingaws.sh update pingone`, which re-derives both. |
| Tier 3: pods pending on memory | Apply the small overlay (above) and close other containers; the default profile wants ~48 Gi of RAM quota. |
| Tier 3: `Refusing local K8 operation on context` | Switch to the local cluster: `kubectl config use-context orbstack`. |

## Where to look for more

- k8s/README.md — full deployment guide (profiles, MLX mode, observability, Privilege gateway)
- docs/superpowers/specs/2026-10-11-share-stack-ghcr-design.md — the sharing architecture (GHCR image split across accounts, secret scanning, recipient setup)
- DOCKER_STARTUP.md / NEW-MACHINE.md — local-Docker details and per-machine gaps